

Teamwork — Session 3: Clear saved, and selectors that hide the store's shape

Context

Work in your team. The class splits into **six teams of four students**.

The saved-opportunities slice now holds the state, and three components read it — each with its own inline arrow function that starts `state.savedOpportunities.savedIds`. That string is knowledge about the store's shape, copied into three files.

There is also nothing in the app that can clear the saved list in one move.

Goal

Add one new action to the slice, and move the store-shape knowledge out of the components and into named selectors that live beside the slice.

What to build

1. **A second reducer.** Add `clearAllSaved` to the existing `savedOpportunitiesSlice`. It empties the saved list. Decide for yourselves what payload it needs, and be ready to justify your answer. Export its generated action creator.
2. **Three named selectors**, defined in the slice file and exported from it:
 - `selectSavedIds(state)` — the array of saved ids;
 - `selectSavedCount(state)` — how many are saved;
 - `selectIsSaved(state, id)` — whether one specific opportunity is saved.
1. **Use each one where it fits.**
 - `Navbar` renders a number, so it should select the count — not the array.
 - `OpportunityCard` needs one yes/no answer about itself, so it should select that.
 - `OpportunitiesSection` genuinely needs the array (the saved-only filter tests membership), so it selects that.
 - After this, no component should contain the string `state.savedOpportunities.savedIds`.
1. **A Clear saved control** in the Opportunities panel header:
 - visible only when at least one opportunity is saved;
 - showing the count, e.g. `Clear saved (3)`;
 - dispatching your new action when pressed;
 - styled so it reads as a secondary action, not a primary button.

Rules

Allowed — everything from today's live coding: `createSlice` reducers, generated action creators, `useSelector`, `useDispatch`, named selector functions, `RootState`.

Not yet (do not use these) — `createAsyncThunk`, `extraReducers`, RTK Query, `createSelector`, custom middleware, persistence libraries.

Do not move `savedOnly` into Redux. It is one panel's own UI state and belongs in `useState` — be ready to explain why.

Test it

- Save three opportunities. The button appears and reads `Clear saved (3)`. The Navbar badge shows 3.
- Press it. Every star resets, the badge disappears, the button disappears, and the saved-only filter shows nothing.
- Save one again: everything comes back consistently.
- The saved-only filter still behaves exactly as before.
- `npm run build` and `npm run lint` pass with 0 errors.

Done when

- All four numbered items above are implemented.
- No component contains the string `state.savedOpportunities.savedIds`.
- `npm run build` and `npm run lint` pass with 0 errors.
- Your team's attempt is committed and pushed to your own branch.

Working time

About **25 minutes**, then the official solution is demonstrated.

Before the solution is shown

Commit and push your team's attempt to your own `student-<your-github-username>` branch **before** the official solution is demonstrated. Pushing your own attempt first is part of the exercise.

Be ready to answer

- What payload does `clearAllSaved` take, and why?
- If the slice key were renamed from `savedOpportunities` to `saved`, how many files would change before your selectors existed? How many after?
- Why does `selectIsSaved` have to be wrapped in an arrow function inside `useSelector`, when `selectSavedCount` does not?